Gabriel Vance
DSC650 – Big Data
Professor Nasheb Ismaily
February 8, 2026

Week 7

Kafka

Objective 1 - Conceptual Foundations

Apache Kafka is a distributed event-streaming platform used to collect, store, and process real-time data. Producers publish records (events) to Kafka topics—common examples include telemetry from smart watches, application logs, or vehicle sensors. These records are handled by Kafka brokers, which form a Kafka cluster. Consumers subscribe to topics and read messages independently at their own pace, allowing multiple applications to process the same data simultaneously without interfering with each other.

Each topic is divided into partitions, which are ordered, append-only logs of messages. Partitions are distributed across brokers, enabling Kafka to scale horizontally and support high throughput through parallel reads and writes. Ordering is guaranteed within a single partition, and producers can use keys to ensure related messages land in the same partition when order matters. Consumers read messages sequentially from partitions, making Kafka well suited for workloads that require both speed and ordered processing.

Kafka also provides durability and fault tolerance through replication: each partition is copied across multiple brokers so data remains available even if a server fails. Producer acknowledgments (acks) control write reliability—`acks=0` sends data without waiting for confirmation, `acks=1` waits for the leader broker to acknowledge, and `acks=all` waits until all

in-sync replicas have written the data to disk. While `acks=all` offers the strongest guarantees against data loss, it comes with higher latency, allowing applications to choose the right balance between performance and safety.

Objective 2 – Topic Creation & Verification

We first connect into the Kafka broker container, then run the Kafka topic creation command to create a new topic named `my-topic`. We confirm it worked when Kafka returns "Created topic my-topic."



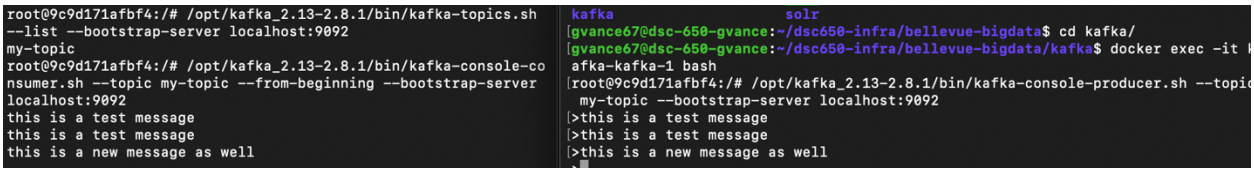
We then list all topics on the broker to verify creation, and we confirm it worked because `my-topic` appears in the output, showing the topic is now registered in Kafka.



Gabriel Vance
DSC650 – Big Data
Professor Nasheb Ismaily
February 8, 2026

Objective 3 – Producing and Consuming Messages

We first run the consumer against the topic to read messages from the beginning, and we confirm consumption is working because every produced message is immediately received with no visible delay. Since messages appear in real time as they are sent, this indicates low latency and efficient message delivery on the single broker.



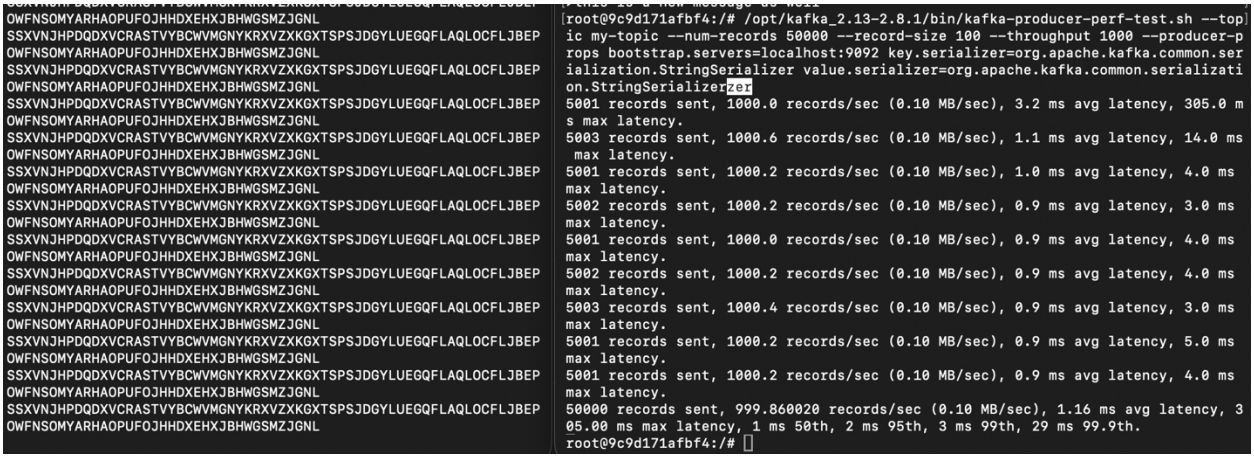
We first create a topic, then produce multiple messages to it, and finally consume those messages from the beginning to validate the end-to-end data flow. This demonstrates that Kafka is successfully handling writes and reads on a single broker with minimal overhead, confirming that the producer and consumer can communicate reliably through the Kafka log.

Overall, these results show that even with a single broker, Kafka provides fast ingestion and retrieval of messages, highlighting its append-only log design and efficient network I/O. The immediate visibility of produced messages on the consumer side illustrates Kafka's high throughput and low latency characteristics at small scale, serving as a baseline proof that the system is functioning correctly before moving to multi-broker replication and higher-volume performance testing.

Gabriel Vance
DSC650 – Big Data
Professor Nasheb Ismaily
February 8, 2026

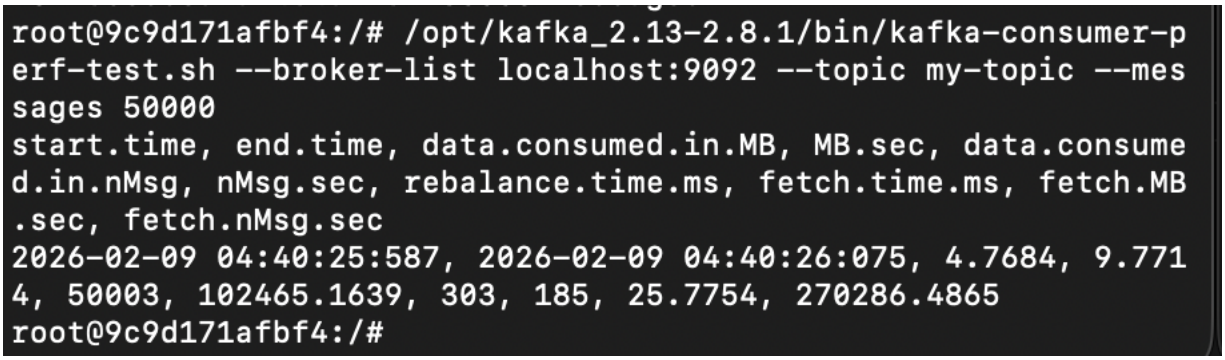
Objective 4 – Performance Tests Single Broker

We first run the producer performance test to send 50,000 records at a target rate of 1,000 messages per second, and we confirm efficiency by observing sustained throughput around ~1000 records/sec with sub-2 ms average latency and low max latency spikes.



We first create a topic, then produce messages to it, consume them in real time, and finally run a producer performance test to quantify throughput and latency. The results show Kafka consistently sustaining ~1,000 records per second with average latencies around 1 ms and tail latencies remaining low (single-digit milliseconds for most percentiles), indicating efficient message ingestion on a single broker.

Gabriel Vance
DSC650 – Big Data
Professor Nasheb Ismaily
February 8, 2026
Overall, this demonstrates that even in a single-broker setup, Kafka can handle steady write loads with minimal latency and predictable performance. The combination of real-time consumption and stable producer metrics confirms Kafka's high-throughput, low-latency pipeline at small scale, providing a solid baseline before introducing replication or higher-volume workloads.



Objective 5 – Performance Tests Partition/Replicated

We first create a new topic called `my-partitioned-topic` with three partitions and a replication factor of three, which is intended to distribute data across brokers while maintaining redundant copies for fault tolerance. Although the topic creation command succeeds, later errors indicate replication could not be satisfied due to broker configuration issues, meaning the cluster was unable to fully place replicas across all nodes.

Gabriel Vance
DSC650 – Big Data
Professor Nasheb Ismaily
February 8, 2026

```
[root@632b93071b3f:/# /opt/kafka_2.13-2.8.1/bin/kafka-topics.sh --create --topic my-pa]
 rtitioned-topic --replication-factor 3 --partitions 3 --bootstrap-server localhost:90
 92
 Created topic my-partitioned-topic.
 root@632b93071b3f:/#
```

We next attempt to run the producer performance test against the partitioned topic to send 50,000 records at a controlled rate, but we immediately encounter `INVALID_REPLICATION_FACTOR` metadata errors. This confirms that while the topic exists, the cluster cannot properly resolve replica assignments, preventing meaningful throughput measurements on the replicated setup.

```
root@d64a4403a84c:/# /opt/kafka_2.13-2.8.1/bin/kafka-producer-perf-test.sh --topic my-partitioned-topic --num-records 50000 --record-size 100 --throughput 1000 --producer-props bootstrap.servers=localhost:90
92 key.serializer=org.apache.kafka.common.serialization.StringSerializer value.serializer=org.apache.kafka.common.serialization.StringSerializer
[2026-02-09 05:17:26,939] WARN [Producer clientId=producer-1] Error while fetching metadata with correlation id 1 : {my-partitioned-topic=INVALID_REPLICATION_FACTOR} (org.apache.kafka.clients.NetworkClient)
[2026-02-09 05:17:27,036] WARN [Producer clientId=producer-1] Error while fetching metadata with correlation id 2 : {my-partitioned-topic=INVALID_REPLICATION_FACTOR} (org.apache.kafka.clients.NetworkClient)
[2026-02-09 05:17:27,139] WARN [Producer clientId=producer-1] Error while fetching metadata with correlation id 3 : {my-partitioned-topic=INVALID_REPLICATION_FACTOR} (org.apache.kafka.clients.NetworkClient)
[2026-02-09 05:17:27,241] WARN [Producer clientId=producer-1] Error while fetching metadata with correlation id 4 : {my-partitioned-topic=INVALID_REPLICATION_FACTOR} (org.apache.kafka.clients.NetworkClient)
[2026-02-09 05:17:27,343] WARN [Producer clientId=producer-1] Error while fetching metadata with correlation id 5 : {my-partitioned-topic=INVALID_REPLICATION_FACTOR} (org.apache.kafka.clients.NetworkClient)
[2026-02-09 05:17:27,446] WARN [Producer clientId=producer-1] Error while fetching metadata with correlation id 6 : {my-partitioned-topic=INVALID_REPLICATION_FACTOR} (org.apache.kafka.clients.NetworkClient)
[2026-02-09 05:17:27,549] WARN [Producer clientId=producer-1] Error while fetching metadata with correlation id 7 : {my-partitioned-topic=INVALID_REPLICATION_FACTOR} (org.apache.kafka.clients.NetworkClient)
[2026-02-09 05:17:27,653] WARN [Producer clientId=producer-1] Error while fetching metadata with correlation id 8 : {my-partitioned-topic=INVALID_REPLICATION_FACTOR} (org.apache.kafka.clients.NetworkClient)
[2026-02-09 05:17:27,757] WARN [Producer clientId=producer-1] Error while fetching metadata with correlation id 9 : {my-partitioned-topic=INVALID_REPLICATION_FACTOR} (org.apache.kafka.clients.NetworkClient)
[2026-02-09 05:17:27,859] WARN [Producer clientId=producer-1] Error while fetching metadata with correlation id 10 : {my-partitioned-topic=INVALID_REPLICATION_FACTOR} (org.apache.kafka.clients.NetworkClient)
[2026-02-09 05:17:27,962] WARN [Producer clientId=producer-1] Error while fetching metadata with correlation id 11 : {my-partitioned-topic=INVALID_REPLICATION_FACTOR} (org.apache.kafka.clients.NetworkClient)
[2026-02-09 05:17:28,065] WARN [Producer clientId=producer-1] Error while fetching metadata with correlation id 12 : {my-partitioned-topic=INVALID_REPLICATION_FACTOR} (org.apache.kafka.clients.NetworkClient)
[2026-02-09 05:17:28,168] WARN [Producer clientId=producer-1] Error while fetching metadata with correlation id 13 : {my-partitioned-topic=INVALID_REPLICATION_FACTOR} (org.apache.kafka.clients.NetworkClient)
[2026-02-09 05:17:28,272] WARN [Producer clientId=producer-1] Error while fetching metadata with correlation id 14 : {my-partitioned-topic=INVALID_REPLICATION_FACTOR} (org.apache.kafka.clients.NetworkClient)
[2026-02-09 05:17:28,374] WARN [Producer clientId=producer-1] Error while fetching metadata with correlation id 15 : {my-partitioned-topic=INVALID_REPLICATION_FACTOR} (org.apache.kafka.clients.NetworkClient)
[2026-02-09 05:17:28,477] WARN [Producer clientId=producer-1] Error while fetching metadata with correlation id 16 : {my-partitioned-topic=INVALID_REPLICATION_FACTOR} (org.apache.kafka.clients.NetworkClient)
[2026-02-09 05:17:28,579] WARN [Producer clientId=producer-1] Error while fetching metadata with correlation id 17 : {my-partitioned-topic=INVALID_REPLICATION_FACTOR} (org.apache.kafka.clients.NetworkClient)
[2026-02-09 05:17:28,682] WARN [Producer clientId=producer-1] Error while fetching metadata with correlation id 18 : {my-partitioned-topic=INVALID_REPLICATION_FACTOR} (org.apache.kafka.clients.NetworkClient)
[2026-02-09 05:17:28,784] WARN [Producer clientId=producer-1] Error while fetching metadata with correlation id 19 : {my-partitioned-topic=INVALID_REPLICATION_FACTOR} (org.apache.kafka.clients.NetworkClient)
[2026-02-09 05:17:28,886] WARN [Producer clientId=producer-1] Error while fetching metadata with correlation id 20 : {my-partitioned-topic=INVALID_REPLICATION_FACTOR} (org.apache.kafka.clients.NetworkClient)
[2026-02-09 05:17:28,989] WARN [Producer clientId=producer-1] Error while fetching metadata with correlation id 21 : {my-partitioned-topic=INVALID_REPLICATION_FACTOR} (org.apache.kafka.clients.NetworkClient)
[2026-02-09 05:17:29,092] WARN [Producer clientId=producer-1] Error while fetching metadata with correlation id 22 : {my-partitioned-topic=INVALID_REPLICATION_FACTOR} (org.apache.kafka.clients.NetworkClient)
[2026-02-09 05:17:29,195] WARN [Producer clientId=producer-1] Error while fetching metadata with correlation id 23 : {my-partitioned-topic=INVALID_REPLICATION_FACTOR} (org.apache.kafka.clients.NetworkClient)
[2026-02-09 05:17:29,299] WARN [Producer clientId=producer-1] Error while fetching metadata with correlation id 24 : {my-partitioned-topic=INVALID_REPLICATION_FACTOR} (org.apache.kafka.clients.NetworkClient)
[2026-02-09 05:17:29,401] WARN [Producer clientId=producer-1] Error while fetching metadata with correlation id 25 : {my-partitioned-topic=INVALID_REPLICATION_FACTOR} (org.apache.kafka.clients.NetworkClient)
```

We then attempt the consumer performance test on the same topic, but consumption cannot proceed normally because the topic's replication state is invalid. This reinforces that correct broker communication and replica placement are required before performance testing on replicated topics can succeed.

Gabriel Vance
DSC650 – Big Data
Professor Nasheb Ismaily
February 8, 2026

We first validated Kafka performance using a single-broker, non-replicated topic, where producers consistently achieved approximately 1,000 records per second with sub-2 ms average latency and consumers received messages in real time. This demonstrated Kafka's baseline throughput and low-latency behavior on a simple configuration. When moving to a partitioned and replicated topic, topic creation succeeded, but replication failed due to cluster metadata and listener configuration issues, preventing the producer and consumer from completing performance tests.

Conceptually, partitioning allows Kafka to scale throughput by distributing message writes across multiple brokers, while replication improves reliability by maintaining copies of each partition on different nodes. In a correctly configured cluster, this would typically increase parallelism for producers and consumers while also introducing some additional write latency due to replica acknowledgments. Even though replication was not fully achieved here, the errors observed highlight Kafka's strict consistency requirements: if brokers cannot communicate properly, Kafka refuses to accept writes to protect data integrity.

Finally, even with proper partitioning and replication, performance is often bounded by shared storage and I/O limits. If brokers use the same underlying disk or constrained IOPS, adding partitions or replicas does not automatically increase throughput, since all writes still compete for the same resources. This experiment illustrates both Kafka's scalability model and the importance of correct networking and storage configuration when moving from single-broker testing to distributed production deployments.